

WEEK-2(Joins & Set Operations)

J. Simran

2520030166

PART 1 — CROSS JOIN

Step 1: Create class

The screenshot shows the SQL Developer interface with the following SQL commands in the script editor:

```
1 • CREATE DATABASE joins_db;  
2 • USE joins_db;
```

The Output pane displays the execution results:

#	Time	Action	Message
✓ 1	12:19:12	CREATE DATABASE joins_db	1 row(s) affected
✗ 2	12:19:18	CREATE DATABASE joins_db	Error Code: 1007. Can't create database 'joins_db'; database...
✓ 3	12:19:18	USE joins_db	0 row(s) affected

Step 2: Create class_info

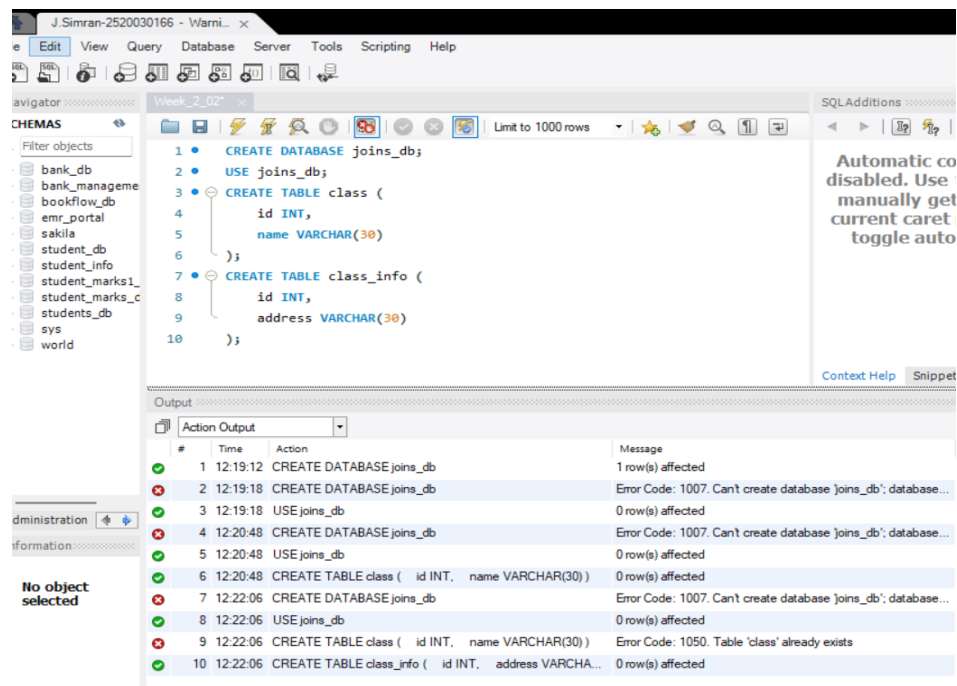
The screenshot shows the SQL Developer interface with the following SQL commands in the script editor:

```
1 • CREATE DATABASE joins_db;  
2 • USE joins_db;  
3 • CREATE TABLE class (  
4     id INT,  
5     name VARCHAR(30)  
6 );
```

The Output pane displays the execution results:

#	Time	Action	Message
✓ 1	12:19:12	CREATE DA	1 row(s) affected
✗ 2	12:19:18	CREATE DATABASE joins_db	Error Code: 1007. Can't create database 'joins_db'; database...
✓ 3	12:19:18	USE joins_db	0 row(s) affected
✗ 4	12:20:48	CREATE DATABASE joins_db	Error Code: 1007. Can't create database 'joins_db'; database...
✓ 5	12:20:48	USE joins_db	0 row(s) affected
✓ 6	12:20:48	CREATE TABLE class (id INT, name VARCHAR(30))	0 row(s) affected

Step 3: Insert class data



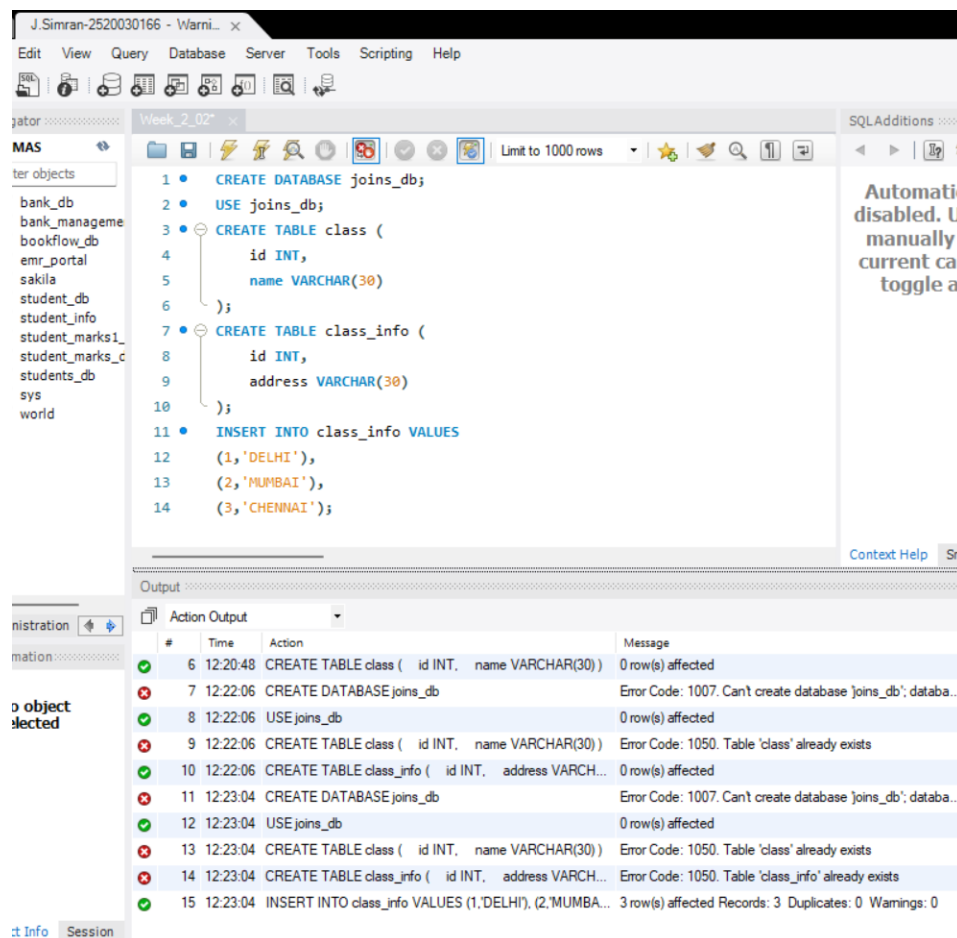
SQL Editor Content:

```
1 CREATE DATABASE joins_db;
2 USE joins_db;
3 CREATE TABLE class (
4     id INT,
5     name VARCHAR(30)
6 );
7 CREATE TABLE class_info (
8     id INT,
9     address VARCHAR(30)
10 );
```

Output:

#	Time	Action	Message
1	12:19:12	CREATE DATABASE joins_db	1 row(s) affected
2	12:19:18	CREATE DATABASE joins_db	Error Code: 1007. Can't create database 'joins_db'; database...
3	12:19:18	USE joins_db	0 row(s) affected
4	12:20:48	CREATE DATABASE joins_db	Error Code: 1007. Can't create database 'joins_db'; database...
5	12:20:48	USE joins_db	0 row(s) affected
6	12:20:48	CREATE TABLE class (id INT, name VARCHAR(30))	0 row(s) affected
7	12:22:06	CREATE DATABASE joins_db	Error Code: 1007. Can't create database 'joins_db'; database...
8	12:22:06	USE joins_db	0 row(s) affected
9	12:22:06	CREATE TABLE class (id INT, name VARCHAR(30))	Error Code: 1050. Table 'class' already exists
10	12:22:06	CREATE TABLE class_info (id INT, address VARCHA...	0 row(s) affected

Step 4: Insert class_info data



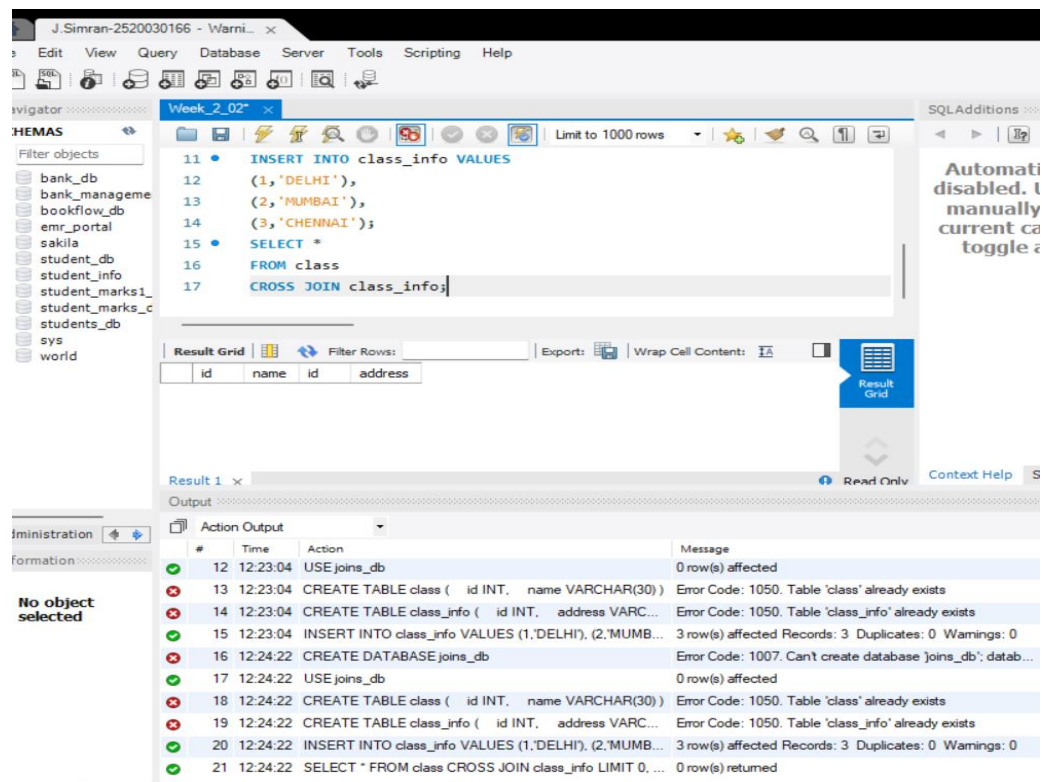
SQL Editor Content:

```
1 CREATE DATABASE joins_db;
2 USE joins_db;
3 CREATE TABLE class (
4     id INT,
5     name VARCHAR(30)
6 );
7 CREATE TABLE class_info (
8     id INT,
9     address VARCHAR(30)
10 );
11 INSERT INTO class_info VALUES
12 (1, 'DELHI'),
13 (2, 'MUMBAI'),
14 (3, 'CHENNAI');
```

Output:

#	Time	Action	Message
6	12:20:48	CREATE TABLE class (id INT, name VARCHAR(30))	0 row(s) affected
7	12:22:06	CREATE DATABASE joins_db	Error Code: 1007. Can't create database 'joins_db'; databa...
8	12:22:06	USE joins_db	0 row(s) affected
9	12:22:06	CREATE TABLE class (id INT, name VARCHAR(30))	Error Code: 1050. Table 'class' already exists
10	12:22:06	CREATE TABLE class_info (id INT, address VARCH...	0 row(s) affected
11	12:23:04	CREATE DATABASE joins_db	Error Code: 1007. Can't create database 'joins_db'; databa...
12	12:23:04	USE joins_db	0 row(s) affected
13	12:23:04	CREATE TABLE class (id INT, name VARCHAR(30))	Error Code: 1050. Table 'class' already exists
14	12:23:04	CREATE TABLE class_info (id INT, address VARCH...	Error Code: 1050. Table 'class_info' already exists
15	12:23:04	INSERT INTO class_info VALUES (1, 'DELHI'), (2, 'MUMBA...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0

Step 5: Run CROSS JOIN



The screenshot shows the SQL Developer interface. The query window contains the following SQL code:

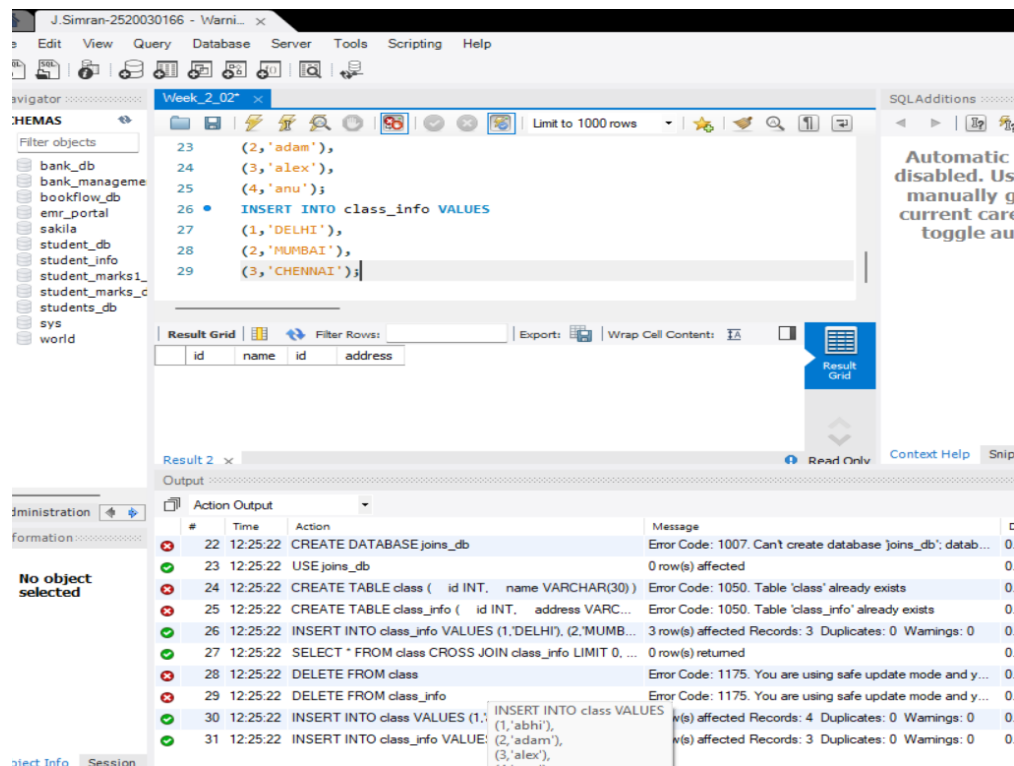
```
11 INSERT INTO class_info VALUES
12 (1,'DELHI'),
13 (2,'MUMBAI'),
14 (3,'CHENNAI');
15 SELECT *
16 FROM class
17 CROSS JOIN class_info;
```

The Action Output window shows the results of the query execution:

#	Time	Action	Message
12	12:23:04	USE joins_db	0 row(s) affected
13	12:23:04	CREATE TABLE class (id INT, name VARCHAR(30))	Error Code: 1050. Table 'class' already exists
14	12:23:04	CREATE TABLE class_info (id INT, address VARC...	Error Code: 1050. Table 'class_info' already exists
15	12:23:04	INSERT INTO class_info VALUES (1,'DELHI'), (2,'MUMB...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
16	12:24:22	CREATE DATABASE joins_db	Error Code: 1007. Can't create database 'joins_db'; datab...
17	12:24:22	USE joins_db	0 row(s) affected
18	12:24:22	CREATE TABLE class (id INT, name VARCHAR(30))	Error Code: 1050. Table 'class' already exists
19	12:24:22	CREATE TABLE class_info (id INT, address VARC...	Error Code: 1050. Table 'class_info' already exists
20	12:24:22	INSERT INTO class_info VALUES (1,'DELHI'), (2,'MUMB...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
21	12:24:22	SELECT * FROM class CROSS JOIN class_info LIMIT 0, ...	0 row(s) returned

PART 2 — INNER JOIN

Step 6: Insert the data needed for INNER JOIN



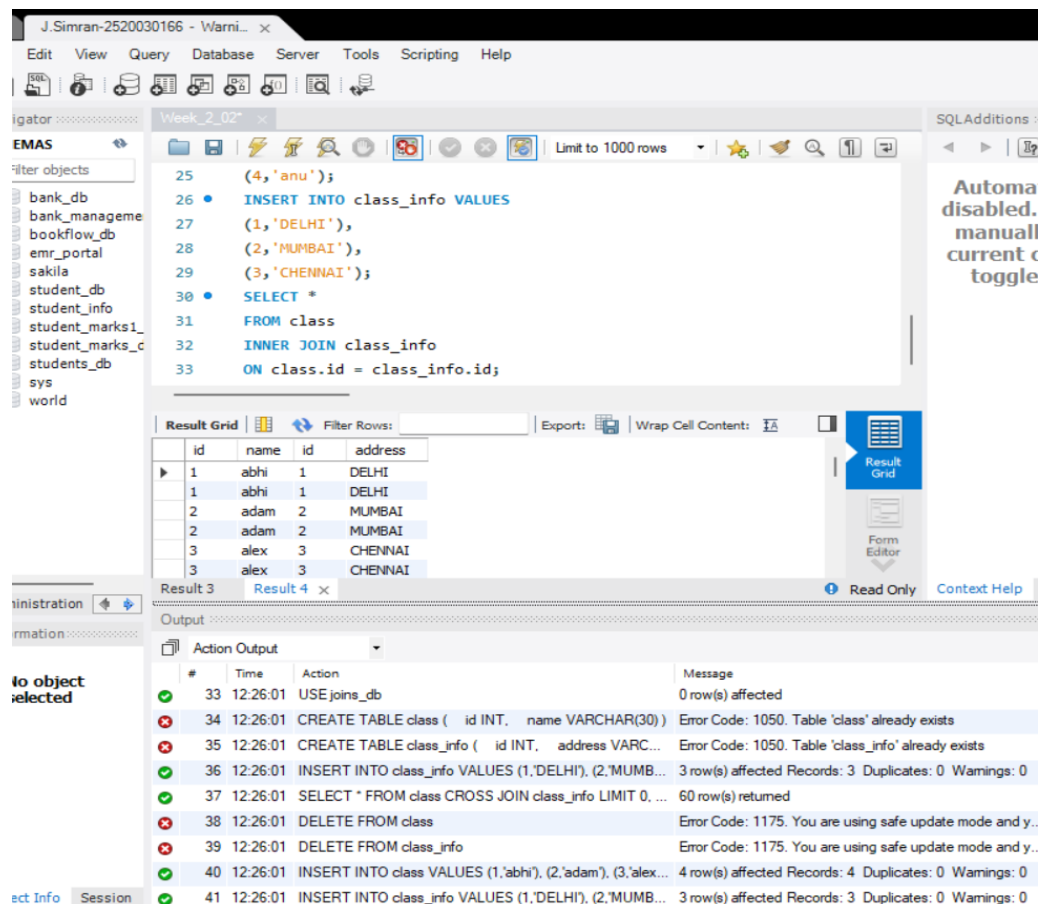
The screenshot shows the SQL Developer interface. The query window contains the following SQL code:

```
23 (2,'adam'),
24 (3,'alex'),
25 (4,'anu');
26 INSERT INTO class_info VALUES
27 (1,'DELHI'),
28 (2,'MUMBAI'),
29 (3,'CHENNAI');
```

The Action Output window shows the results of the query execution:

#	Time	Action	Message
22	12:25:22	CREATE DATABASE joins_db	Error Code: 1007. Can't create database 'joins_db'; datab...
23	12:25:22	USE joins_db	0 row(s) affected
24	12:25:22	CREATE TABLE class (id INT, name VARCHAR(30))	Error Code: 1050. Table 'class' already exists
25	12:25:22	CREATE TABLE class_info (id INT, address VARC...	Error Code: 1050. Table 'class_info' already exists
26	12:25:22	INSERT INTO class_info VALUES (1,'DELHI'), (2,'MUMB...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
27	12:25:22	SELECT * FROM class CROSS JOIN class_info LIMIT 0, ...	0 row(s) returned
28	12:25:22	DELETE FROM class	Error Code: 1175. You are using safe update mode and y...
29	12:25:22	DELETE FROM class_info	Error Code: 1175. You are using safe update mode and y...
30	12:25:22	INSERT INTO class VALUES (1,'abhi'), (2,'adam'), (3,'alex'),	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
31	12:25:22	INSERT INTO class_info VALUE:	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0

Step 7: INNER JOIN



SQL Developer interface showing the query editor and the result grid.

Query Editor:

```
25 (4,'anu');
26 • INSERT INTO class_info VALUES
27 (1,'DELHI'),
28 (2,'MUMBAI'),
29 (3,'CHENNAI');
30 • SELECT *
31 FROM class
32 INNER JOIN class_info
33 ON class.id = class_info.id;
```

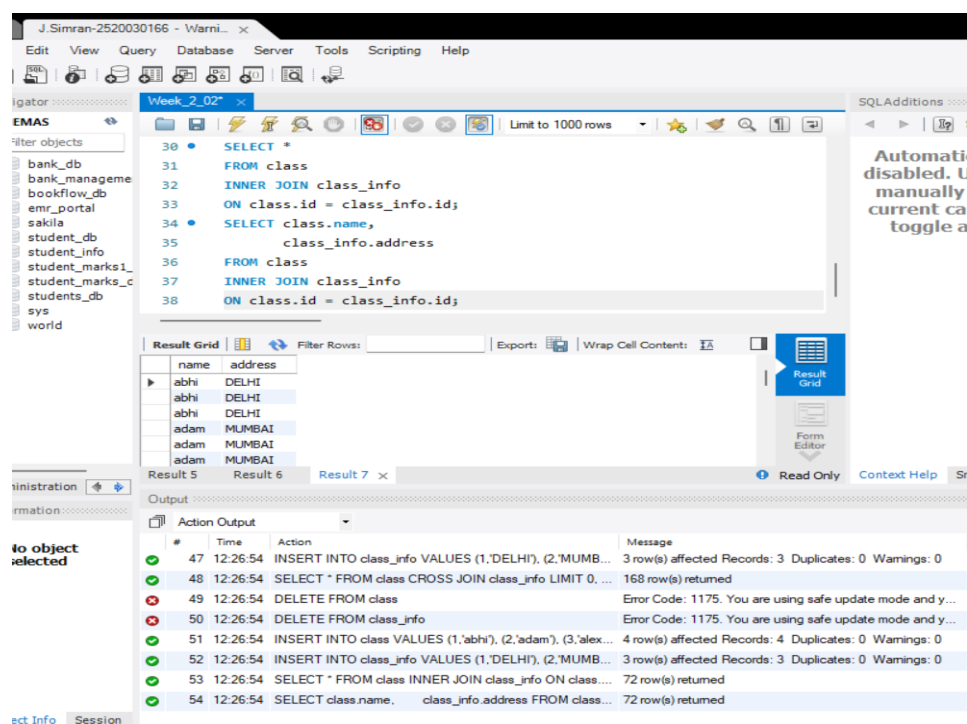
Result Grid:

	id	name	id	address
▶	1	abhi	1	DELHI
	1	abhi	1	DELHI
	2	adam	2	MUMBAI
	2	adam	2	MUMBAI
	3	alex	3	CHENNAI
	3	alex	3	CHENNAI

Output:

#	Time	Action	Message
33	12:26:01	USE joins_db	0 row(s) affected
34	12:26:01	CREATE TABLE class (id INT, name VARCHAR(30))	Error Code: 1050. Table 'class' already exists
35	12:26:01	CREATE TABLE class_info (id INT, address VARC...	Error Code: 1050. Table 'class_info' already exists
36	12:26:01	INSERT INTO class_info VALUES (1,'DELHI'), (2,'MUMB...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
37	12:26:01	SELECT * FROM class CROSS JOIN class_info LIMIT 0, ...	60 row(s) returned
38	12:26:01	DELETE FROM class	Error Code: 1175. You are using safe update mode and y...
39	12:26:01	DELETE FROM class_info	Error Code: 1175. You are using safe update mode and y...
40	12:26:01	INSERT INTO class VALUES (1,'abhi'), (2,'adam'), (3,'alex...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0
41	12:26:01	INSERT INTO class_info VALUES (1,'DELHI'), (2,'MUMB...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0

Step 8: INNER JOIN — only name and address



SQL Developer interface showing the query editor and the result grid.

Query Editor:

```
30 • SELECT *
31 FROM class
32 INNER JOIN class_info
33 ON class.id = class_info.id;
34 • SELECT class.name,
35 class_info.address
36 FROM class
37 INNER JOIN class_info
38 ON class.id = class_info.id;
```

Result Grid:

	name	address
▶	abhi	DELHI
	abhi	DELHI
	abhi	DELHI
	adam	MUMBAI
	adam	MUMBAI
	adam	MUMBAI

Output:

#	Time	Action	Message
47	12:26:54	INSERT INTO class_info VALUES (1,'DELHI'), (2,'MUMB...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
48	12:26:54	SELECT * FROM class CROSS JOIN class_info LIMIT 0, ...	168 row(s) returned
49	12:26:54	DELETE FROM class	Error Code: 1175. You are using safe update mode and y...
50	12:26:54	DELETE FROM class_info	Error Code: 1175. You are using safe update mode and y...
51	12:26:54	INSERT INTO class VALUES (1,'abhi'), (2,'adam'), (3,'alex...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0
52	12:26:54	INSERT INTO class_info VALUES (1,'DELHI'), (2,'MUMB...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
53	12:26:54	SELECT * FROM class INNER JOIN class_info ON class....	72 row(s) returned
54	12:26:54	SELECT class.name, class_info.address FROM class...	72 row(s) returned

PART 3 — NATURAL JOIN

Step 9: NATURAL JOIN

The screenshot shows the SQL Developer interface with a query window titled "Week_2_02". The query is as follows:

```
33 ON class.id = class_info.id;
34 SELECT class.name,
35        class_info.address
36 FROM class
37 INNER JOIN class_info
38 ON class.id = class_info.id;
39 SELECT *
40 FROM class
41 NATURAL JOIN class_info;
```

The "Result Grid" shows the output of the NATURAL JOIN query, displaying columns "id", "name", and "address". The results are:

id	name	address
1	abhi	DELHI
1	abhi	DELHI
1	abhi	DELHI
1	abhi	DELHI
2	adam	MUMBAI
2	adam	MUMBAI

The "Action Output" pane shows the execution log, including the following messages:

- 60 12:27:31 SELECT * FROM class CROSS JOIN class_info LIMIT 0, ... 324 row(s) returned
- 61 12:27:31 DELETE FROM class Error Code: 1175. You are using safe update mode and y...
- 62 12:27:31 DELETE FROM class_info Error Code: 1175. You are using safe update mode and y...
- 63 12:27:31 INSERT INTO class VALUES (1,'abhi'), (2,'adam'), (3,'alex... 4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0
- 64 12:27:31 INSERT INTO class_info VALUES (1,'DELHI'), (2,'MUMB... 3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
- 65 12:27:31 SELECT * FROM class INNER JOIN class_info ON class... 120 row(s) returned
- 66 12:27:31 SELECT class.name, class_info.address FROM class... 120 row(s) returned
- 67 12:27:31 SELECT * FROM class NATURAL JOIN class_info LIMIT... 120 row(s) returned

PART 4 — LEFT JOIN

Step 10: Add extra records

The screenshot shows the SQL Developer interface with a query window titled "Week_2_02". The query is as follows:

```
39 SELECT *
40 FROM class
41 NATURAL JOIN class_info;
42 INSERT INTO class VALUES
43 (5,'ashish');
44
45 INSERT INTO class_info VALUES
46 (7,'NOIDA'),
47 (8,'PANIPAT');
```

The "Result Grid" shows the output of the query, displaying columns "id", "name", and "address". The results are:

id	name	address
1	abhi	DELHI
1	abhi	DELHI
1	abhi	DELHI
1	abhi	DELHI
2	adam	MUMBAI
2	adam	MUMBAI

The "Action Output" pane shows the execution log, including the following messages:

- 75 12:28:10 DELETE FROM class_info Error Code: 1175. You are using safe update mode and y...
- 76 12:28:10 INSERT INTO class VALUES (1,'abhi'), (2,'adam'), (3,'alex... 4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0
- 77 12:28:10 INSERT INTO class_info VALUES (1,'DELHI'), (2,'MUMB... 3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
- 78 12:28:10 SELECT * FROM class INNER JOIN class_info ON class... 180 row(s) returned
- 79 12:28:10 SELECT class.name, class_info.address FROM class... 180 row(s) returned
- 80 12:28:10 SELECT * FROM class NATURAL JOIN class_info LIMIT... 180 row(s) returned
- 81 12:28:10 INSERT INTO class VALUES (5,'ashish') 1 row(s) affected
- 82 12:28:10 INSERT INTO class_info VALUES (7,'NOIDA'), (8,'PANIP... 2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0

Step 11: LEFT JOIN

The screenshot shows the SQL Developer interface with a query window titled 'Week_2_02'. The query is as follows:

```
43 (5,'ashish');
44
45 INSERT INTO class_info VALUES
46 (7,'NOIDA'),
47 (8,'PANIPAT');
48 SELECT *
49 FROM class
50 LEFT OUTER JOIN class_info
51 ON class.id = class_info.id;
```

The 'Result Grid' shows the following data:

id	name	id	address
1	abhi	1	DELHI
1	abhi	1	DELHI
1	abhi	1	DELHI
2	adam	2	MUMBAI
2	adam	2	MUMBAI
2	adam	2	MUMBAI

The 'Output' pane shows the execution log with the following messages:

- 91 12:28:45 INSERT INTO class VALUES (1,'abhi'), (2,'adam'), (3,'alex') 4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0
- 92 12:28:45 INSERT INTO class_info VALUES (1,'DELHI'), (2,'MUMB... 3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
- 93 12:28:45 SELECT * FROM class INNER JOIN class_info ON class... 252 row(s) returned
- 94 12:28:45 SELECT class.name, class_info.address FROM class... 252 row(s) returned
- 95 12:28:45 SELECT * FROM class NATURAL JOIN class_info LIMIT... 252 row(s) returned
- 96 12:28:45 INSERT INTO class VALUES (5,'ashish') 1 row(s) affected
- 97 12:28:45 INSERT INTO class_info VALUES (7,'NOIDA'), (8,'PANIP... 2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
- 98 12:28:45 SELECT * FROM class LEFT OUTER JOIN class_info O... 260 row(s) returned

Step 12: Find students without an address

The screenshot shows the SQL Developer interface with a query window titled 'Week_2_02'. The query is as follows:

```
48 SELECT *
49 FROM class
50 LEFT OUTER JOIN class_info
51 ON class.id = class_info.id;
52 SELECT *
53 FROM class
54 LEFT JOIN class_info
55 ON class.id = class_info.id
56 WHERE class_info.id IS NULL;
```

The 'Result Grid' shows the following data:

id	name	id	address
4	anu	NULL	NULL
4	anu	NULL	NULL
4	anu	NULL	NULL
4	anu	NULL	NULL
4	anu	NULL	NULL
5	ashish	NULL	NULL

The 'Output' pane shows the execution log with the following messages:

- 108 12:29:24 INSERT INTO class_info VALUES (1,'DELHI'), (2,'MUMB... 3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0
- 109 12:29:24 SELECT * FROM class INNER JOIN class_info ON class... 346 row(s) returned
- 110 12:29:24 SELECT class.name, class_info.address FROM class... 346 row(s) returned
- 111 12:29:24 SELECT * FROM class NATURAL JOIN class_info ON class.id = class_info.id... 346 row(s) returned
- 112 12:29:24 INSERT INTO class VALUES (5,'ashish') 1 row(s) affected
- 113 12:29:24 INSERT INTO class_info VALUES (7,'NOIDA'), (8,'PANIP... 2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
- 114 12:29:24 SELECT * FROM class LEFT OUTER JOIN class_info O... 346 row(s) returned
- 115 12:29:24 SELECT * FROM class LEFT JOIN class_info ON class.id... 10 row(s) returned

PART 5 — RIGHT JOIN

Step 13: RIGHT JOIN

The screenshot shows the SQL Developer interface with a query window titled 'Week_2_02*'. The query is as follows:

```
52 SELECT *
53 FROM class
54 LEFT JOIN class_info
55 ON class.id = class_info.id
56 WHERE class_info.id IS NULL;
57 SELECT *
58 FROM class
59 RIGHT OUTER JOIN class_info
60 ON class.id = class_info.id;
```

The 'Result Grid' shows the results of the second query (lines 57-60). It displays a table with columns 'id', 'name', 'id', and 'address'. The data is as follows:

id	name	id	address
1	abhi	1	DELHI
1	abhi	1	DELHI
1	abhi	1	DELHI
1	abhi	1	DELHI
1	abhi	1	DELHI
1	abhi	1	DELHI
1	abhi	1	DELHI

The 'Output' pane shows the execution log with the following messages:

- 126 12:29:58 SELECT * FROM class INNER JOIN class_info ON class.... 432 row(s) returned
- 127 12:29:58 SELECT class.name, class_info.address FROM class... 432 row(s) returned
- 128 12:29:58 SELECT * FROM class NATURAL JOIN class_info LIMIT... 432 row(s) returned
- 129 12:29:58 INSERT INTO class VALUES (5,'ashish') 1 row(s) affected
- 130 12:29:58 INSERT INTO class_info VALUES (7,'NOIDA'), (8,'PANIPAT... 2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
- 131 12:29:58 SELECT * FROM class LEFT OUTER JOIN class_info O... 444 row(s) returned
- 132 12:29:58 SELECT * FROM class LEFT JOIN class_info ON class.id... 12 row(s) returned
- 133 12:29:58 SELECT * FROM class RIGHT OUTER JOIN class_info ... 440 row(s) returned

Step 14: Find addresses without students

The screenshot shows the SQL Developer interface with a query window titled 'Week_2_02*'. The query is as follows:

```
57 SELECT *
58 FROM class
59 RIGHT OUTER JOIN class_info
60 ON class.id = class_info.id;
61 SELECT *
62 FROM class
63 RIGHT JOIN class_info
64 ON class.id = class_info.id
65 WHERE class_info.id IS NULL;
```

The 'Result Grid' shows the results of the second query (lines 61-65). It displays a table with columns 'id', 'name', 'id', and 'address'. The data is as follows:

id	name	id	address
NULL	NULL	7	NOIDA
NULL	NULL	8	PANIPAT
NULL	NULL	7	NOIDA
NULL	NULL	8	PANIPAT
NULL	NULL	7	NOIDA
NULL	NULL	8	PANIPAT

The 'Output' pane shows the execution log with the following messages:

- 145 12:30:37 SELECT class.name, class_info.address FROM class... 540 row(s) returned
- 146 12:30:37 SELECT * FROM class NATURAL JOIN class_info LIMIT... 540 row(s) returned
- 147 12:30:37 INSERT INTO class VALUES (5,'ashish') 1 row(s) affected
- 148 12:30:37 INSERT INTO class_info VALUES (7,'NOIDA'), (8,'PANIPAT... 2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
- 149 12:30:37 SELECT * FROM class LEFT OUTER JOIN class_info O... 554 row(s) returned
- 150 12:30:37 SELECT * FROM class LEFT JOIN class_info ON class.id... 14 row(s) returned
- 151 12:30:37 SELECT * FROM class RIGHT OUTER JOIN class_info ... 550 row(s) returned
- 152 12:30:37 SELECT * FROM class RIGHT JOIN class_info ON class.... 10 row(s) returned

PART 6 — FULL OUTER JOIN

Step 15: FULL OUTER JOIN

The screenshot shows the SQL Developer interface with a query window titled "Week_2_02". The query is as follows:

```
66 SELECT *
67 FROM class
68 LEFT JOIN class_info
69 ON class.id = class_info.id
70 UNION
71 SELECT *
72 FROM class
73 RIGHT JOIN class_info
74 ON class.id = class_info.id;
```

The "Result Grid" shows the output of the query:

id	name	id	address
3	alex	3	CHENNAI
4	anu	NULL	NULL
5	ashish	NULL	NULL
NULL	NULL	7	NOIDA
NULL	NULL	8	PANIPAT

The "Output" pane shows the execution log with the following messages:

- 165 12:32:12 SELECT * FROM class NATURAL JOIN class_info LIMIT... 660 row(s) returned
- 166 12:32:12 INSERT INTO class VALUES (5,'ashish') 1 row(s) affected
- 167 12:32:12 INSERT INTO class_info VALUES (7,'NOIDA'), (8,'PANIPAT') 2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
- 168 12:32:12 SELECT * FROM class LEFT OUTER JOIN class_info O... 676 row(s) returned
- 169 12:32:12 SELECT * FROM class LEFT JOIN class_info ON class.id... 16 row(s) returned
- 170 12:32:12 SELECT * FROM class RIGHT OUTER JOIN class_info ... 672 row(s) returned
- 171 12:32:12 SELECT * FROM class RIGHT JOIN class_info ON class... 12 row(s) returned
- 172 12:32:12 SELECT * FROM class LEFT JOIN class_info ON class.id... 7 row(s) returned

Step 16: Find only unmatched records

The screenshot shows the SQL Developer interface with a query window titled "Week_2_02". The query is as follows:

```
71 SELECT *
72 FROM class
73 RIGHT JOIN class_info
74 ON class.id = class_info.id;
75 SELECT *
76 FROM class
77 LEFT JOIN class_info
78 ON class.id = class_info.id
79 WHERE class_info.id IS NULL;
```

The "Result Grid" shows the output of the query:

id	name	id	address
4	anu	NULL	NULL
5	ashish	NULL	NULL
NULL	NULL	7	NOIDA
NULL	NULL	8	PANIPAT

The "Output" pane shows the execution log with the following messages:

- 186 12:33:26 INSERT INTO class VALUES (5,'ashish') 1 row(s) affected
- 187 12:33:26 INSERT INTO class_info VALUES (7,'NOIDA'), (8,'PANIPAT') 2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
- 188 12:33:26 SELECT * FROM class LEFT OUTER JOIN class_info O... 810 row(s) returned
- 189 12:33:26 SELECT * FROM class LEFT JOIN class_info ON class.id... 18 row(s) returned
- 190 12:33:26 SELECT * FROM class RIGHT OUTER JOIN class_info ... 806 row(s) returned
- 191 12:33:26 SELECT * FROM class RIGHT JOIN class_info ON class... 14 row(s) returned
- 192 12:33:26 SELECT * FROM class LEFT JOIN class_info ON class.id... 7 row(s) returned
- 193 12:33:26 SELECT * FROM class LEFT JOIN class_info ON class.id... 4 row(s) returned

PART 7 — UNION

Step 17: Create two tables

The screenshot shows the SQL Developer interface with the following SQL code in the editor:

```
81 SELECT *
82 FROM class
83 RIGHT JOIN class_info
84 ON class.id = class_info.id
85 WHERE class.id IS NULL;
86 CREATE TABLE first_table(
87     id INT,
88     name VARCHAR(30)
89 );
90 CREATE TABLE second_table(
```

The Result Grid shows the following data:

	id	name	id	address
4	anu	NULL	NULL	
5	ashish	NULL	NULL	
	NULL	NULL	7	NOIDA
	NULL	NULL	8	PANIPAT

The Action Output pane shows the following messages:

#	Time	Action	Message
232	12:36:32	SELECT * FROM class LEFT OUTER JOIN class_info O...	1000 row(s) returned
233	12:36:32	SELECT * FROM class LEFT JOIN class_info ON class.id...	22 row(s) returned
234	12:36:32	SELECT * FROM class RIGHT OUTER JOIN class_info ...	1000 row(s) returned
235	12:36:32	SELECT * FROM class RIGHT JOIN class_info ON class....	18 row(s) returned
236	12:36:32	SELECT * FROM class LEFT JOIN class_info ON class.id...	7 row(s) returned
237	12:36:32	SELECT * FROM class LEFT JOIN class_info ON class.id...	4 row(s) returned
238	12:36:32	CREATE TABLE first_table(id INT, name VARCHA...	0 row(s) affected
239	12:36:32	CREATE TABLE second_table(id INT, name VARC...	0 row(s) affected

Step 18: Insert data

The screenshot shows the SQL Developer interface with the following SQL code in the editor:

```
93 );
94 INSERT INTO first_table VALUES
95 (1,'abhi'),
96 (2,'adam');
97
98 INSERT INTO second_table VALUES
99 (2,'adam'),
100 (3,'chester');
```

The Result Grid shows the following data:

	id	name	id	address
4	anu	NULL	NULL	
5	ashish	NULL	NULL	
	NULL	NULL	7	NOIDA
	NULL	NULL	8	PANIPAT

The Action Output pane shows the following messages:

#	Time	Action	Message
43	13:26:20	SELECT * FROM class RIGHT OUTER JOIN class_info ON ...	1000 row(s) returned
44	13:26:20	SELECT * FROM class RIGHT JOIN class_info ON class.id ...	22 row(s) returned
45	13:26:20	SELECT * FROM class LEFT JOIN class_info ON class.id = ...	7 row(s) returned
46	13:26:20	SELECT * FROM class LEFT JOIN class_info ON class.id = ...	4 row(s) returned
47	13:26:20	CREATE TABLE first_table(id INT, name VARCHAR(3...	Error Code: 1050. Table 'first_table' already exists
48	13:26:20	CREATE TABLE second_table(id INT, name VARCHA...	Error Code: 1050. Table 'second_table' already exists
49	13:26:20	INSERT INTO first_table VALUES (1,'abhi'), (2,'adam')	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
50	13:26:20	INSERT INTO second_table VALUES (2,'adam'), (3,'chester')	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0

Step 19: UNION

MySQL Workbench - J. Simran-2520030166 - Warni...

File Edit View Query Database Server Tools Scripting Help

Navigator: Week_2_02*

HEMAS

Filter objects

- bank_db
- bank_managem...
- bookflow_db
- emr_portal
- joins_db
 - Tables
 - Views
 - Stored Proc...
 - Functions
- sakila
- student_db
- student_info
- student_marks1
- student_marks_c
- students_db
- sys
- world

SQL Editor:

```
96 (2,'adam');
97
98 INSERT INTO second_table VALUES
99 (2,'adam'),
100 (3,'chester');
101 SELECT * FROM first_table
102 UNION
103 SELECT * FROM second_table;
```

Result Grid:

	id	name
▶	1	abhi
	2	adam
	3	chester

Output:

#	Time	Action	Message
69	13:26:56	SELECT * FROM class RIGHT JOIN class_info ON class.id = ...	24 row(s) returned
70	13:26:56	SELECT * FROM class LEFT JOIN class_info ON class.id = ...	7 row(s) returned
71	13:26:56	SELECT * FROM class LEFT JOIN class_info ON class.id = ...	4 row(s) returned
72	13:26:56	CREATE TABLE first_table(id INT, name VARCHAR(3...	Error Code: 1050. Table 'first_table' already exists
73	13:26:56	CREATE TABLE second_table(id INT, name VARCHA...	Error Code: 1050. Table 'second_table' already exists
74	13:26:56	INSERT INTO first_table VALUES (1,'abhi'), (2,'adam')	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
75	13:26:56	INSERT INTO second_table VALUES (2,'adam'), (3,'chester')	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
76	13:26:56	SELECT * FROM first_table UNION SELECT * FROM secon...	3 row(s) returned

Step 20: UNION names

MySQL Workbench - J. Simran-2520030166 - Warni...

File Edit View Query Database Server Tools Scripting Help

Navigator: Week_2_02*

SCHEMAS

Filter objects

- bank_db
- bank_managem...
- bookflow_db
- emr_portal
- joins_db
 - Tables
 - Views
 - Stored Proc...
 - Functions
- sakila
- student_db
- student_info
- student_marks1
- student_marks_c
- students_db
- sys
- world

SQL Editor:

```
99 (2,'adam'),
100 (3,'chester');
101 SELECT * FROM first_table
102 UNION
103 SELECT * FROM second_table;
104 SELECT name FROM first_table
105 UNION
106 SELECT name FROM second_table;
```

Result Grid:

	name
▶	abhi
	adam
	chester

Output:

#	Time	Action	Message
123	13:28:48	SELECT * FROM class LEFT JOIN class_info ON class.id = ...	7 row(s) returned
124	13:28:48	SELECT * FROM class LEFT JOIN class_info ON class.id = ...	4 row(s) returned
125	13:28:48	CREATE TABLE first_table(id INT, name VARCHAR(3...	Error Code: 1050. Table 'first_table' already exists
126	13:28:48	CREATE TABLE second_table(id INT, name VARCHA...	Error Code: 1050. Table 'second_table' already exists
127	13:28:48	INSERT INTO first_table VALUES (1,'abhi'), (2,'adam')	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
128	13:28:48	INSERT INTO second_table VALUES (2,'adam'), (3,'chester')	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
129	13:28:48	SELECT * FROM first_table UNION SELECT * FROM secon...	3 row(s) returned
130	13:28:48	SELECT name FROM first_table UNION SELECT name FR...	3 row(s) returned

PART 8 — UNION ALL

Step 21

The screenshot shows the SQL Developer interface with a query window titled "Week_2_02". The query is as follows:

```
102 UNION
103 SELECT * FROM second_table;
104 SELECT name FROM first_table
105 UNION
106 SELECT name FROM second_table;
107 SELECT * FROM first_table
108 UNION ALL
109 SELECT * FROM second_table;
```

The "Result Grid" shows the output of the query, which is a list of rows with columns "id" and "name". The rows are:

id	name
1	abhi
2	adam
1	abhi
2	adam
1	abhi
2	adam
1	abhi

The "Output" pane shows the execution log, including the following messages:

- 151 13:30:45 SELECT * FROM class LEFT JOIN class_info ON class id = ... 4 row(s) returned
- 152 13:30:45 CREATE TABLE first_table(id INT, name VARCHAR(3... Error Code: 1050. Table 'first_table' already exists
- 153 13:30:45 CREATE TABLE second_table(id INT, name VARCHA... Error Code: 1050. Table 'second_table' already exists
- 154 13:30:45 INSERT INTO first_table VALUES (1,'abhi'), (2,'adam') 2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
- 155 13:30:45 INSERT INTO second_table VALUES (2,'adam'), (3,'chester') 2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
- 156 13:30:45 SELECT * FROM first_table UNION SELECT * FROM secon... 3 row(s) returned
- 157 13:30:45 SELECT name FROM first_table UNION SELECT name FR... 3 row(s) returned
- 158 13:30:45 SELECT * FROM first_table UNION ALL SELECT * FROM s... 20 row(s) returned

Step 22: Count all records

The screenshot shows the SQL Developer interface with a query window titled "Week_2_02". The query is as follows:

```
109 SELECT * FROM second_table;
110 SELECT COUNT(*)
111 FROM
112 (
113 SELECT * FROM first_table
114 UNION ALL
115 SELECT * FROM second_table
116 ) AS
```

The "Result Grid" shows the output of the query, which is a single row with the value 24.

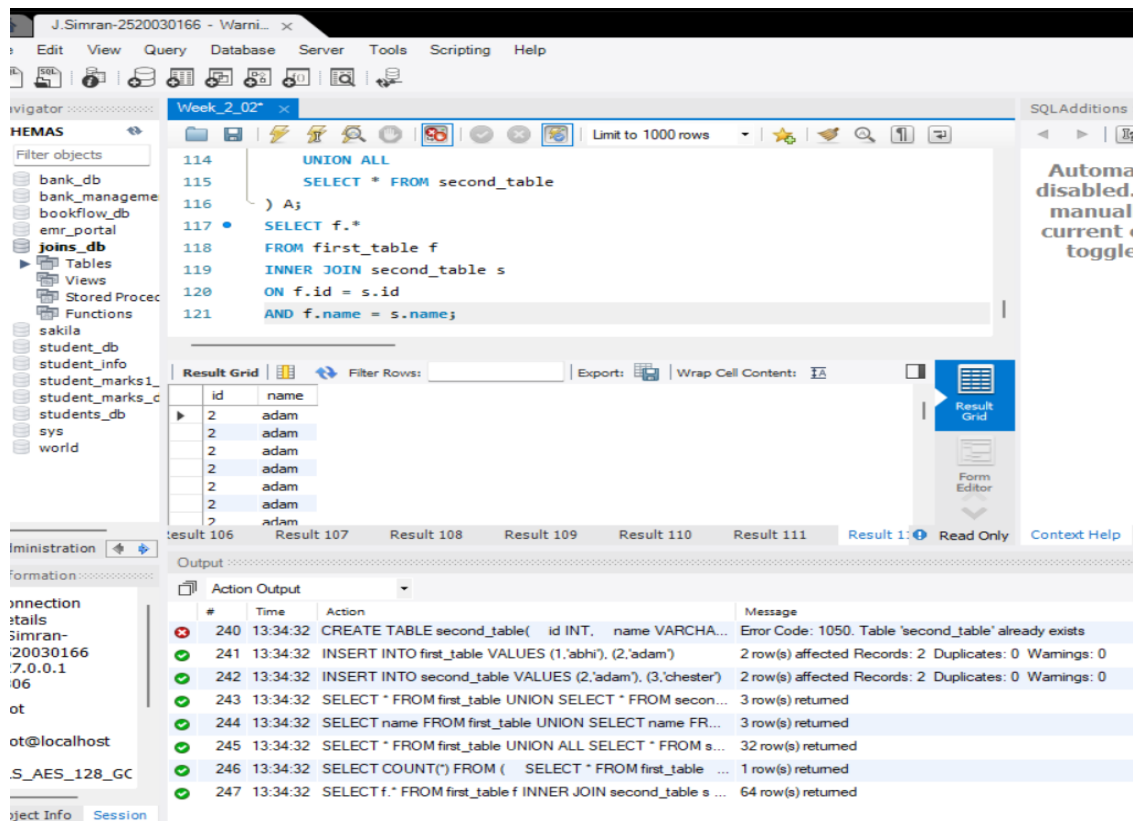
COUNT(*)
24

The "Output" pane shows the execution log, including the following messages:

- 180 13:31:41 CREATE TABLE first_table(id INT, name VARCHAR(3... Error Code: 1050. Table 'first_table' already exists
- 181 13:31:41 CREATE TABLE second_table(id INT, name VARCHA... Error Code: 1050. Table 'second_table' already exists
- 182 13:31:41 INSERT INTO first_table VALUES (1,'abhi'), (2,'adam') 2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
- 183 13:31:41 INSERT INTO second_table VALUES (2,'adam'), (3,'chester') 2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
- 184 13:31:41 SELECT * FROM first_table UNION SELECT * FROM secon... 3 row(s) returned
- 185 13:31:41 SELECT name FROM first_table UNION SELECT name FR... 3 row(s) returned
- 186 13:31:41 SELECT * FROM first_table UNION ALL SELECT * FROM s... 24 row(s) returned
- 187 13:31:41 SELECT COUNT(*) FROM (SELECT * FROM first_table ... 1 row(s) returned

PART 9 — INTERSECT

Step 23



The screenshot shows the SQL Developer interface with a query window titled "Week_2_02". The query is as follows:

```
114 UNION ALL
115 SELECT * FROM second_table
116 ) AS;
117 SELECT f.*
118 FROM first_table f
119 INNER JOIN second_table s
120 ON f.id = s.id
121 AND f.name = s.name;
```

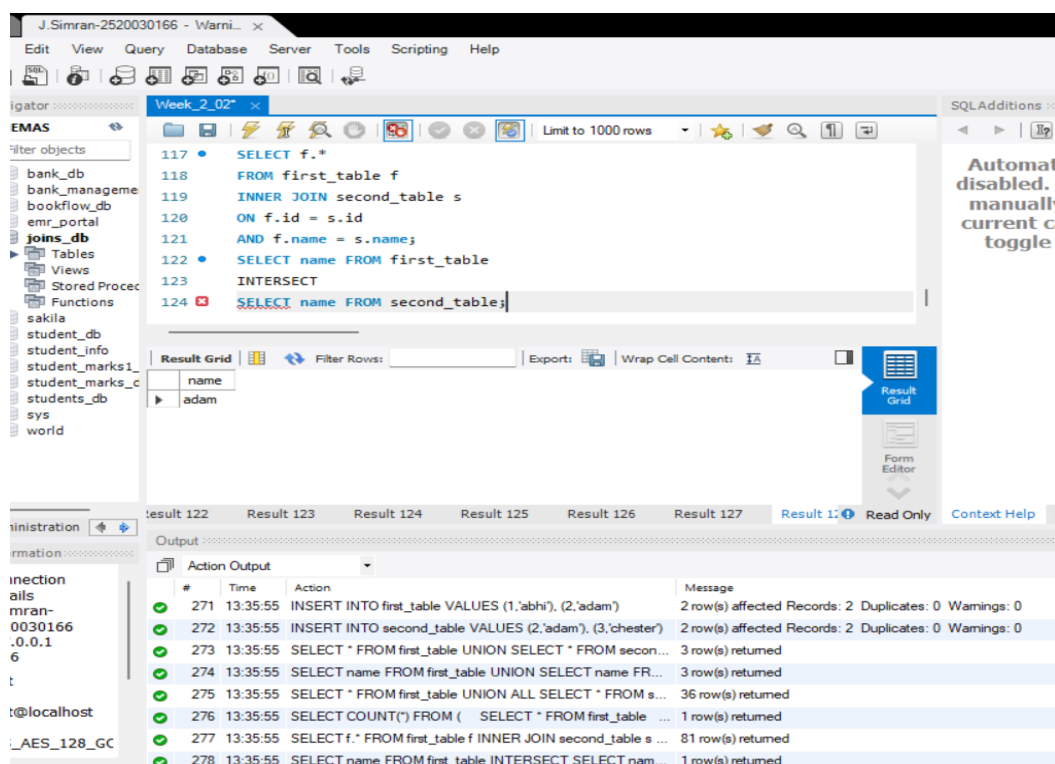
The result grid displays the following data:

id	name
2	adam
2	adam
2	adam
2	adam
2	adam
2	adam
2	adam
2	adam
2	adam
2	adam

The output pane shows the execution of the query with the following messages:

#	Time	Action	Message
240	13:34:32	CREATE TABLE second_table(id INT, name VARCHAR...	Error Code: 1050. Table 'second_table' already exists
241	13:34:32	INSERT INTO first_table VALUES (1,'abhi'), (2,'adam')	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
242	13:34:32	INSERT INTO second_table VALUES (2,'adam'), (3,'chester')	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
243	13:34:32	SELECT * FROM first_table UNION SELECT * FROM second...	3 row(s) returned
244	13:34:32	SELECT name FROM first_table UNION SELECT name FR...	3 row(s) returned
245	13:34:32	SELECT * FROM first_table UNION ALL SELECT * FROM s...	32 row(s) returned
246	13:34:32	SELECT COUNT(*) FROM (SELECT * FROM first_table ...	1 row(s) returned
247	13:34:32	SELECT f.* FROM first_table f INNER JOIN second_table s ...	64 row(s) returned

Step 24: INTERSECT names



The screenshot shows the SQL Developer interface with a query window titled "Week_2_02". The query is as follows:

```
117 SELECT f.*
118 FROM first_table f
119 INNER JOIN second_table s
120 ON f.id = s.id
121 AND f.name = s.name;
122 SELECT name FROM first_table
123 INTERSECT
124 SELECT name FROM second_table;
```

The result grid displays the following data:

name
adam

The output pane shows the execution of the query with the following messages:

#	Time	Action	Message
271	13:35:55	INSERT INTO first_table VALUES (1,'abhi'), (2,'adam')	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
272	13:35:55	INSERT INTO second_table VALUES (2,'adam'), (3,'chester')	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
273	13:35:55	SELECT * FROM first_table UNION SELECT * FROM second...	3 row(s) returned
274	13:35:55	SELECT name FROM first_table UNION SELECT name FR...	3 row(s) returned
275	13:35:55	SELECT * FROM first_table UNION ALL SELECT * FROM s...	36 row(s) returned
276	13:35:55	SELECT COUNT(*) FROM (SELECT * FROM first_table ...	1 row(s) returned
277	13:35:55	SELECT f.* FROM first_table f INNER JOIN second_table s ...	81 row(s) returned
278	13:35:55	SELECT name FROM first_table INTERSECT SELECT nam...	1 row(s) returned

PART 10 — MINUS

Step 25

The screenshot shows the SQL Developer interface with a query window titled 'Week_2_02'. The query is as follows:

```
123 FROM first_table f
124 INNER JOIN second_table s
125 ON f.name = s.name;SELECT f.*
126 FROM first_table f
127 LEFT JOIN second_table s
128 ON f.id = s.id
129 AND f.name = s.name
130 WHERE s.id IS NULL;
```

The 'Result Grid' shows the following data:

id	name
1	abhi
1	abhi
1	abhi
1	abhi
1	abhi
1	abhi
1	abhi

The 'Output' window shows the following messages:

#	Time	Action	Message
367	13:38:20	INSERT INTO second_table VALUES (2,'adam'), (3,'chester')	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0
368	13:38:20	SELECT * FROM first_table UNION SELECT * FROM second_table	3 row(s) returned
369	13:38:20	SELECT name FROM first_table UNION SELECT name FROM second_table	3 row(s) returned
370	13:38:20	SELECT * FROM first_table UNION ALL SELECT * FROM second_table	48 row(s) returned
371	13:38:20	SELECT COUNT(*) FROM (SELECT * FROM first_table UNION ALL SELECT * FROM second_table)	1 row(s) returned
372	13:38:20	SELECT f.* FROM first_table f INNER JOIN second_table s ON f.name = s.name	144 row(s) returned
373	13:38:20	SELECT DISTINCT f.name FROM first_table f INNER JOIN second_table s ON f.name = s.name	1 row(s) returned
374	13:38:20	SELECT f.* FROM first_table f LEFT JOIN second_table s ON f.id = s.id AND f.name = s.name	12 row(s) returned

Step 26: MINUS names

The screenshot shows the SQL Developer interface with a query window titled 'Week_2_02'. The query is as follows:

```
128 ON f.id = s.id
129 AND f.name = s.name
130 WHERE s.id IS NULL;
131 SELECT f.name
132 FROM first_table f
133 LEFT JOIN second_table s
134 ON f.name = s.name
135 WHERE s.name IS NULL;
```

The 'Result Grid' shows the following data:

name
abhi
abhi
abhi
abhi
abhi
abhi
abhi

The 'Output' window shows the following messages:

#	Time	Action	Message
400	13:40:42	SELECT * FROM first_table UNION SELECT * FROM second_table	3 row(s) returned
401	13:40:42	SELECT name FROM first_table UNION SELECT name FROM second_table	3 row(s) returned
402	13:40:42	SELECT * FROM first_table UNION ALL SELECT * FROM second_table	52 row(s) returned
403	13:40:42	SELECT COUNT(*) FROM (SELECT * FROM first_table UNION ALL SELECT * FROM second_table)	1 row(s) returned
404	13:40:42	SELECT f.* FROM first_table f INNER JOIN second_table s ON f.name = s.name	169 row(s) returned
405	13:40:42	SELECT DISTINCT f.name FROM first_table f INNER JOIN second_table s ON f.name = s.name	1 row(s) returned
406	13:40:42	SELECT f.* FROM first_table f LEFT JOIN second_table s ON f.id = s.id AND f.name = s.name	13 row(s) returned
407	13:40:42	SELECT f.name FROM first_table f LEFT JOIN second_table s ON f.id = s.id AND f.name = s.name	13 row(s) returned

PART 11 — Advanced Questions

Step 27: Matching addresses

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
132 FROM first_table f
133 LEFT JOIN second_table s
134 ON f.name = s.name
135 WHERE s.name IS NULL;
136 SELECT c.id,c.name,ci.address
137 FROM class c
138 INNER JOIN class_info ci
139 ON c.id=ci.id;
```

The Result Grid shows the output of the query:

id	name	address
1	abhi	DELHI
1	abhi	DELHI
1	abhi	DELHI
1	abhi	DELHI
2	adam	MUMBAI
2	adam	MUMBAI
2	adam	MUMBAI

The Output pane shows the Action Output for the query:

#	Time	Action	Message
434	13:41:42	SELECT name FROM first_table UNION SELECT name FR...	3 row(s) returned
435	13:41:42	SELECT * FROM first_table UNION ALL SELECT * FROM s...	56 row(s) returned
436	13:41:42	SELECT COUNT(*) FROM (SELECT * FROM first_table ...	1 row(s) returned
437	13:41:42	SELECT f.* FROM first_table f INNER JOIN second_table s ...	196 row(s) returned
438	13:41:42	SELECT DISTINCT f.name FROM first_table f INNER JOIN ...	1 row(s) returned
439	13:41:42	SELECT f.* FROM first_table f LEFT JOIN second_table s O...	14 row(s) returned
440	13:41:42	SELECT f.name FROM first_table f LEFT JOIN second_tabl...	14 row(s) returned
441	13:41:42	SELECT c.id,c.name,ci.address FROM class c INNER JOIN...	1000 row(s) returned

Step 28: Address availability

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
142 CASE
143 WHEN ci.address IS NULL
144 THEN 'Address Missing'
145 ELSE 'Address Available'
146 END AS Status
147 FROM class c
148 LEFT JOIN class_info ci
149 ON c.id=ci.id;
```

The Result Grid shows the output of the query:

id	name	Status
1	abhi	Address Available
1	abhi	Address Available
2	adam	Address Available
2	adam	Address Available
2	adam	Address Available
2	adam	Address Available
2	adam	Address Available

The Output pane shows the Action Output for the query:

#	Time	Action	Message
469	13:42:22	SELECT * FROM first_table UNION ALL SELECT * FROM s...	60 row(s) returned
470	13:42:22	SELECT COUNT(*) FROM (SELECT * FROM first_table ...	1 row(s) returned
471	13:42:22	SELECT f.* FROM first_table f INNER JOIN second_table s ...	225 row(s) returned
472	13:42:22	SELECT DISTINCT f.name FROM first_table f INNER JOIN ...	1 row(s) returned
473	13:42:22	SELECT f.* FROM first_table f LEFT JOIN second_table s O...	15 row(s) returned
474	13:42:22	SELECT f.name FROM first_table f LEFT JOIN second_tabl...	15 row(s) returned
475	13:42:22	SELECT c.id,c.name,ci.address FROM class c INNER JOIN...	1000 row(s) returned
476	13:42:22	SELECT c.id, c.name, CASE WHEN ci.addr...	1000 row(s) returned